

# Python I. beadandó feladat

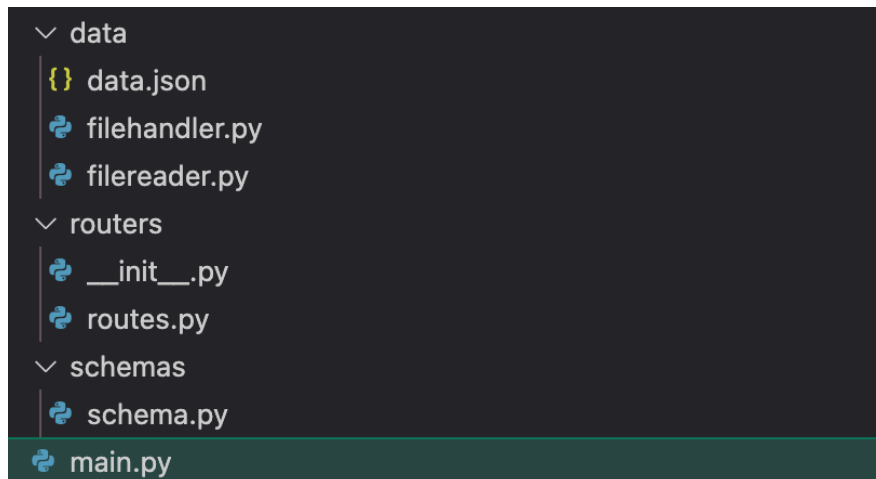
## ELTE IK PNYFT – 2026 B feladat

### REST API készítése FastAPI alapokon

A beadandó feladatban elkészítendő API felhasználók kezelését teszi lehetővé. Tartalmazza azok bolthoz rendelését, a kosár hozzáadását egy adott vásárlóhoz, valamint a vásárló kosarába termékeket elhelyezését az API végpontok segítségével.

A kosárba elhelyezett termékeket lehet módosítani, vagy törölni. A megvásárolt termékek összeértékét, vagyis a fizetendő összeget is le lehet kérdezni egy adott vásárló kosarában található termékek árai alapján. Az API a fentiekén kívül lehetőséget biztosít egy adott vásárló adatainak a lekérdezésére, az összes vásárló megjelenítésére, vagy egy adott vásárlói kosár tartalmának a megjelenítésére. Ezekkel a funkciókkal lehetőségünk nyílik egy webbolt elkészítésére, ami már a felhasználói felületet is tartalmazza (A felhasználói felületet nem kell elkészíteni, de amennyiben az API-hoz készül ilyen felület, az extra pontokat adhat a megoldáshoz.). Az API a végpontok által kezelt adatokat a data.json nevű fájlban tárolja, amely fájl tartalmát lehetőségünk menteni és visszatölteni az erre a célra kialakított endpoint-ok meghívásával. A feladat tehát, amit meg kell oldanunk, egy FastAPI implementációjának a kiegészítése. A kód egy része már meg van írva a mellékelt könyvtárakban. Az előre elkészített fájlokban instrukciók vannak az adott modul funkcióinak a megírásához és véglegesítéséhez.

### A mellékelt fájlok tartalma



- A data.json a JSON az adatok tárolására szolgál. Ebben elhelyeztünk néhány teszt adatot, amelyeket fel lehet használni az alkalmazás kipróbálásához.
- A filehandler.py és a filereader.py a JSON adatok fájlból történő beolvasására és fájlba írására használható.
- A routes.py tartalmazza az alkalmazás végpontjait. Ezeket a függvényeket kell kiegészíteni a megfelelő funkciókkal és paraméterekkel.
- A schemas.py a JSON adatok kliens és szerver közötti mozgatásához, valamint a JSON adatok kezelésére alkalmas osztályokat tartalmazza. Ezeket az osztályokat

ki kell egészíteni a megfelelő mezőkkel a JSON adatok alapján. Ügyelni kell az adatok helyességére is a fájlban leírtak alapján.

- A main.py tartalmazza a FastAPI main modulját, amelyet futtatni kell az alkalmazás elindításához a uvicorn python csomag segítségével.

## Feladatok

FastAPI 0.1.0 OAS 3.1  
/openapi.json

|         |                                |   |
|---------|--------------------------------|---|
| default |                                | ^ |
| POST    | /adduser Adduser               | ▼ |
| POST    | /addshoppingbag Addshoppingbag | ▼ |
| POST    | /additem Additem               | ▼ |
| PUT     | /updateitem Updateitem         | ▼ |
| DELETE  | /deleteitem Deleteitem         | ▼ |
| GET     | /user Users                    | ▼ |
| GET     | /users Users                   | ▼ |
| GET     | /shoppingbag Shoppingbag       | ▼ |
| GET     | /getusertotal Getusertotal     | ▼ |
| GET     | / Route                        | ▼ |
| Schemas |                                | ^ |

- Az itt látható végpontokat kell elkészítenünk **(a feladat leírás végén az endpoint-ok listája még kiegészül!)**.
- Az adduser segítségével egy felhasználót tudunk hozzáadni a webshophoz. **(5 pont)**
- Az addshoppingbag egy kosarat rendel egy felhasználóhoz. **(5 pont)**
- Az additem egy terméket ad az adott felhasználó kosarához. **(5 pont)**
- Az updateitem módosítja az adott termék attribútumait egy felhasználó kosarában. **(10 pont)**
- A deleteitem töröl egy terméket a kosárból. **(5 pont)**
- A user egy adott felhasználó adatait adja vissza. **(5 pont)**
- A users visszaadja az összes felhasználót. **(5 pont)**
- A shoppingbag egy kosárban található összes termék adatait adja vissza. **(5 pont)**
- A getusertotal egy vásárló kosarában található termékek értékét adja vissza. **(5 pont)**
- A Route az üzlet nevét adja vissza. Ez a funkció már implementálva van a main.py fájlban.

Az endpoint-ok teszteléséhez Indítsuk el a REST API-t és nézzük meg listát a FastAPI által automatikusan generált UI felületén. Az API elindításakor a rendszer generálni fog egy URL-t és egy portot rendel a futó programhoz. Ennek az URL-nek a kiegészítésével elérjük az alkalmazás /doc endpointjával a webes felületet. Az alkalmazás elindítása:

***uvicorn mainfile:neve:appneve --reload***

Az alkalmazás webes felületén (/docs) a fenti feladatokat el lehet végezni. Minden egyes funkció a router fájlban leírt módon viselkedik, és az ott meghatározott adatokat adja vissza Szabványos JSON formátumban, a JsonResponse függvénnyel, státus kóddal együtt.

A végpontok megírása során az alábbi szabályokat kell betartani **(20 pont)**:

- Minden végpontnál adjuk meg a response\_modell értékét (típus).
- Ügyeljünk a típusok megadására a függvényekben is.
- Ahol lehet, a függvények visszatérési értéke JsonResponse() legyen
- Minden függvény tartalmazzon hibakezelést, hiba esetén dobjon egy HTTPException-t és a megfelelő status code-ot (ahol ennek van értelme).
- A függvények a JSON adatok mentéséhez és visszaolvasásához a filehandler.py és a filereader.py fájlt használják. Tegyük elérhetővé ezeket az alkalmazásban! Az adatokat a data.json fájlba kell menteni.
- A HTTP válaszok minden esetben tartalmazzák a megfelelő status code-ot, pl 404 - Not found, vagy 200 – OK.

## Az alkalmazás használata

A main.py fájl a REST API main modulja indítja el. A futtatásához telepíteni kell a pip csomagkezelővel a uvicorn és a fastapi csomagokat:

**pip install uvicorn, fastapi** vagy **pip3 install uvicorn, fastapi**

Ezután az alkalmazás a következő paranccsal futtatható a terminálban:

**uvicorn main:app --reload --port 9000**

A port nem kötelező opció.

A futó alkalmazás a következő URL-en érhető el:

**127.0.0.1:9000**

**(5 pont)**

A webes UI pedig az alábbi URL használatával:

**127.0.0.1:9000/docs**

A dokumentáció pedig itt:

**127.0.0.1:9000/redoc**

Az adatok kezelésére használható osztályokat a megadott schema alapján ki kell dolgozni. A schema.py tartalmazza azok küldésére és fogadására készített osztályokat. Az osztályokban az adatok legyenek validálva! **(15 pont)**

- Az int adatok nem lehetnek negatívak.

- Az email mező csak e-mail formátumot fogadhat el.
- Hiba esetén ValueErrort kell dobni és lehetőség szerint a kliens oldalon is jelezni kell a hibát.

## A végpontok implementációja

Az alábbi végpontokat kell kidolgozni:

Felhasználó hozzáadása a bolthoz. A bemenet egy User típus, a visszatérési érték a felvitt felhasználó rekordja.

```
@routers.post('/adduser', response_model=User)
def adduser(user: User) -> User:
    pass
```

Kosár hozzáadása egy User-hez. A bemenő paraméter a felhasználó azonosítója, a visszatérési érték a következő szöveg: „Sikeres kosár hozzárendelés.”

```
@routers.post('/addshoppingbag')
def addshoppingbag(userid: int) -> str:
    pass
```

Termék berakása a felhasználó kosarába. A bemenő paraméterek a felhasználó azonosítója és a termék. A visszatérési érték a kosár tartalma.

```
@routers.post('/additem', response_model=Basket)
def additem(userid: int, item: Item) -> Basket:
    pass
```

Egy adott termék attribútumainak módosítása. A bemenő paraméter a felhasználó azonosítója, valamint a termék azonosítója és új attribútumai. Termék cserével is megoldható a feladat. A visszatérési érték a kosár tartalma.

```
@routers.put('/updateitem')
def updateitem(userid: id, itemid: int, updateItem: Item) -> Basket:
    pass
```

Egy termék törlése az adott felhasználó kosarából. A bemenő paraméterek a felhasználó azonosítója, valamint a termék azonosítója. A visszatérési érték a kosár tartalma.

```
@routers.delete('/deleteitem')
def deleteitem(userid: int, itemid: int) -> Basket:
    pass
```

Egy adott felhasználó adatainak a megjelenítése. A bemenő paraméter a felhasználó azonosítója, a visszatérési érték a felhasználó rekordja.

```
@routers.get('/user')  
def user(userid: int) -> User:  
    pass
```

Az összes felhasználó lekérdezése az adatbázisból. Nincs bemenő paraméter és a kosarak nem kerülnek megjelenítésre. A visszatérési érték a felhasználók listája.

```
@routers.get('/users')  
def users() -> list[User]:  
    pass
```

Egy adott felhasználó kosarának megjelenítése. A paramétere a felhasználó azonosítója. A visszatérési érték a termékek.

```
@routers.get('/shoppingbag')  
def shoppinbag(userid: int) -> list[Item]:  
    pass
```

Egy adott felhasználó kosarában lévő termékek értékét adja vissza. A bemenő paramétere a felhasználó azonosítója, a visszatérési érték az összeg.

```
@routers.get('/getusertotal')  
def getusertotal(userid: int) -> float:  
    pass
```

Az API rendelkezik egy mentést segítő endpointtal, ahol a mentésre szánt fájl neve, valamint a mentést tartalmazó cél fájl a paraméterek. A funkciókhoz tartozó fájl a JSON adatokat kezelő fájlok könyvtárában legyen elhelyezve. A mentés és a visszatöltés lehetőség szerint a korábban bemutatott Schema alapján valósuljon meg, hogy minél kevesebb adatvesztés, vagy adathiba forduljon elő. **(10 pont)**

```
@routers.post('/save')  
def save(source:str, dest:str)  
    pass
```

Az API rendelkezik egy mentett adatok visszatöltését segítő endpointtal, ahol a visszatöltendő fájl neve, valamint a cél fájl a paraméterek. **(10 pont)**

```
@routers.post('/reload')  
def save(dest:str, source:str)  
    pass
```

**Tesztelés:** Az Endpointokat - a beépített tesztelési eljárások mellett - egy külső szoftverrel is teszteljük (erre alkalmas lehet a Postman webes alkalmazás, de más szoftver is elfogadható.) **A tesztelésről egy rövid dokumentációt is készítsünk, amelyet a védés során be kell mutatni az alkalmazással együtt.**

**Az endpoint-ok indokolt esetben módosíthatók, vagy ki is lehet őket egészíteni.**

**Kérdés esetén a gyakorlatvezetőhöz, vagy az előadás vezetőjéhez lehet fordulni.**

A feladat értékeléséhez minden végpontnak működnie kell. A pontszám a megvalósítás minőségétől is függ, vagyis, ha egy végpont működik, de a megadott feltételeknek nem felel meg, akkor kevesebb pontot ér. A tesztelés is követelménye a feladat megoldásának!

Az elérhető maximális pontszám: **110 pont**  
**Jó munkát!**